

```

        list = new List<Pair<T, U>>();
    }

    /// <summary>
    /// Indexer for the map. Takes Ts and returns Us.
    /// </summary>
    public U this[T t]
    {
        get
        {
            foreach (Pair<T, U> p in list)
            {
                if (p.Fst.Equals(t)) { return p.Snd; }
            }
            return default(U);
        }
        set
        {
            foreach (Pair<T, U> p in list)
            {
                if (p.Fst.Equals(t))
                {
                    list.Remove(p);
                    break;
                }
            }
            list.Add(new Pair<T, U>(t, value));
        }
    }
}

```

Finally, like C++, we can specify how to convert from one type to another via conversion operators. Note conversion operators must be marked implicit or explicit. Explicit conversions must be signified by casts.

```

public struct MyType
{
    private int X { get; set; }
    public MyType(int x) : this()
    {

```